Lab12 Counter UVM Testbench — Complete Guide

All concepts explained with diagrams | Changes from Lab11 highlighted

1. Overview

Lab12 extends the Lab11 counter UVM testbench with the five key UVM infrastructure features used in every industry-grade verification environment: configuration objects, scoreboards, functional coverage, virtual sequencers, and the factory type-override mechanism. The Device Under Test (DUT) is unchanged — the same 8-bit loadable up/down counter.

After completing Lab12 the testbench can run three fully independent tests from the same compiled image by simply changing the +UVM_TESTNAME plusarg:

```
make questa_batch TEST=counter_test          # directed 6-transaction test
make questa_batch TEST=counter_random_test   # factory override → random
make questa_batch TEST=counter_virtual_test  # virtual sequence pattern
```

2. What Changed from Lab11 to Lab12

The table below lists every file in the Lab12 testbench. Files marked NEW did not exist in Lab11. Files marked UPDATED were modified. Files marked UNCHANGED are identical to Lab11.

File	Status	Lab12 Concept Added	Key Change
counter.sv	UNCHANGED	—	DUT RTL — not modified
counter_if.sv	UNCHANGED	—	Interface tasks — not modified
counter_transaction.sv	UNCHANGED	—	Sequence item — minor convert2string update
counter_config.sv	NEW	Configuration object	num_transactions, enable_coverage, do_copy, do_print
counter_driver.sv	UPDATED	Analysis port (stimulus)	Added ap; publishes transaction BEFORE driving DUT
counter_monitor.sv	UNCHANGED	—	Already had ap — not modified
counter_agent.sv	UNCHANGED	—	Not modified
counter_scoreboard.sv	NEW	Dual-imp scoreboard	stim_imp + resp_imp; expected FIFO queue; golden model
counter_coverage.sv	NEW	Functional coverage	uvm_component + explicit uvm_analysis_imp_cov; covergroup test_type × value × cycles
counter_virtual_sequencer.sv	NEW	Virtual sequencer	Holds counter_seqr handle; assigned in connect_phase
counter_env.sv	UPDATED	Env wires all Lab12 parts	Creates sb, cov, vseqr; connects all analysis paths
counter_sequence.sv	EXTRACTED	Factory-override base	Moved out of counter_test.sv into its own

		class	file
counter_random_sequence.sv	**NEW**	Constrained-random stimulus	Reads num_transactions from config; randomize()
counter_virtual_sequence.sv	**NEW**	Virtual sequence	Casts m_sequencer; runs directed then random
counter_test.sv	**UPDATED**	Base test creates config	Config creation + config_db deposit; env creation
counter_random_test.sv	**NEW**	Factory type override	set_type_override before super.build_phase
counter_virtual_test.sv	**NEW**	Virtual sequence test	Starts vseq on env.vseqr
counter_tb_top.sv	**UPDATED**	run_test() without argument	Creates config; run_test() uses +UVM_TESTNAME
counter_pkg.sv	**UPDATED**	Includes all new files	Include list in dependency order
Makefile	**UPDATED**	TEST= variable	make questa_batch TEST=<testname>

Key concept introductio

1. m_sequencer (member sequencer)

- What it is: A built-in handle of type `uvm_sequencer_base` inside every UVM sequence. - Purpose: Provides generic sequencer methods (`get_full_name()`, `wait_for_grant()`, etc.). - Limitation: Cannot access sequencer-specific properties/methods because it's a base class handle. - Always available: Yes, without any macro.

2. p_sequencer (pointer sequencer) –

What it is: A user-defined handle that points to the actual, concrete sequencer type (e.g., `my_sequencer`). –

Purpose: Allows a sequence to directly access custom signals, variables, or methods of the sequencer it runs on. –

How to use: Declare with `uvm_declare_p_sequencer(my_sequencer)` inside the sequence. The macro automatically casts `m_sequencer` to the derived type. –

Availability: Only after using the macro.

Key relationship: `p_sequencer` is a type-safe, downcasted version of `m_sequencer`.

3. Virtual Sequencer –

What it is: A sequencer that does not drive any interface. It contains handles to multiple physical sequencers (e.g., `axi_seqr`, `gpio_seqr`). –

Purpose: Acts as a router or container – it lets a virtual sequence coordinate stimulus across different sequencers.

- Creation: Derived from `uvm_sequencer` (unparameterized). Typically built in the test or environment. –

Connection: In the `connect_phase`, assign the physical sequencer handles from the agents to the virtual sequencer.

3. Testbench Architecture

The diagram below shows the complete Lab12 testbench hierarchy. Components shaded in grey existed in Lab11. Components in bold are new in Lab12.

Figure 1 — Complete Lab12 testbench hierarchy

```
┌──────────────────────────────────────────────────────────────────────────┐
│  counter_tb_top  (module)                                                │
│   counter_if intf ──────────────────────────────────┐                    │
│   counter dut (RTL)  ◀──────────────────────────────┘                    │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  uvm_test_top  (UVM root)                                        │    │
│  │                                                                  │    │
│  │  ┌──────────────────────────────────────────────────────────┐    │    │
│  │  │  counter_test  [or random_test / virtual_test]           │    │    │
│  │  │                                                          │    │    │
│  │  │  counter_config ◀── config_db ──▶ counter_tb_top         │    │    │
│  │  │                                                          │    │    │
│  │  │  ┌──────────────────────────────────────────────┐        │    │    │
│  │  │  │  counter_env                                 │        │    │    │
│  │  │  │                                              │        │    │    │
│  │  │  │  ┌──────────────────────┐                    │        │    │    │
│  │  │  │  │  counter_agent       │                    │        │    │    │
│  │  │  │  │   counter_sequencer ──▶ driver ──▶ DUT    │        │    │    │
│  │  │  │  │                   ap ──┐                 │        │    │    │
│  │  │  │  │   counter_monitor  ap ──┤                 │        │    │    │
│  │  │  │  └──────────────────────┘                    │        │    │    │
│  │  │  │        │  (stim)  │  (resp)                  │        │    │    │
│  │  │  │        ▼          ▼                          │        │    │    │
│  │  │  │  counter_scoreboard  ◀── NEW                 │        │    │    │
│  │  │  │        │                                     │        │    │    │
│  │  │  │  driver.ap ──────────▶ counter_coverage ◀ NEW │        │    │    │
│  │  │  │                                              │        │    │    │
│  │  │  │  counter_virtual_sequencer ◀── NEW           │        │    │    │
│  │  │  │   .counter_seqr ──▶ agent.sequencer          │        │    │    │
│  │  │  └──────────────────────────────────────────────┘        │    │    │
│  │  └──────────────────────────────────────────────────────────┘    │    │
│  └──────────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────┘
```

The key structural difference from Lab11: the environment now has four components instead of one. The agent's analysis ports are connected to the scoreboard and coverage collector in counter_env::connect_phase.

4. Configuration Object (counter_config.sv — NEW)

4.1 What Is a Config Object?

In Lab11, settings like "how many transactions to run" were hard-coded inside test classes. A configuration object centralises all knobs in one place. The test creates a config, fills in its fields, deposits it in uvm_config_db, and every component that needs a setting retrieves it independently — without direct references between test and component.

Figure 2 — Config object flow from tb_top through test to env and sequences

```
  counter_tb_top                        counter_test
  ______________                        ____________

  counter_config cfg;                   counter_config cfg;
  cfg.num_transactions = 20;            // try to get from db
  cfg.enable_coverage  = 1;             if (!config_db::get(..., cfg))
  config_db::set(null,"*","cfg",cfg);   //   create default
                     |                  config_db::set(this,"*","cfg",cfg);
                     ▼                              |
             config_db  ◄──────────────────────────┘
                     |
                     ▼
  counter_env::build_phase              counter_random_sequence::body()
    config_db::get(this,"","cfg",cfg)    config_db::get(seqr,"","cfg",cfg)
```

4.2 Fields and Their Purpose

Field	Type / Default	Purpose
num_transactions	int = 20	How many transactions counter_random_sequence generates. Reduces/increases regression run time.
enable_coverage	bit = 1	When 1, counter_env creates counter_coverage in build_phase and connects it in connect_phase. Set to 0 to skip coverage collection and save simulation time.
use_random_test	bit = 0	Informational flag. counter_random_test sets it to 1 so other components can query which test mode is active.

4.3 do_copy and do_print Hooks

counter_config extends uvm_object, which provides copy() and print() built-in methods. Those methods call virtual hooks do_copy() and do_print() that the subclass must override to handle its custom fields:

Figure 3 — do_copy copies all fields; do_print uses print_field() (UVM 1.1d compatible)

```
  do_copy(rhs):                                do_print(printer):
    super.do_copy(rhs);                          super.do_print(printer);
    $cast(other, rhs);                           // Use print_field() — UVM 1.1d
built-in API
    num_transactions = other.num_transactions; // (print_field_int exists only
in UVM 1.2)
    enable_coverage  = other.enable_coverage;  printer.print_field(
    use_random_test  = other.use_random_test;      "num_transactions",
num_transactions,
```

```
                                                        32, UVM_DEC);
                                                    printer.print_field(
                                                        "enable_coverage",
enable_coverage,
                                                        1, UVM_BIN);
```

5. Driver Analysis Port (counter_driver.sv — UPDATED)

5.1 What Was Added

Lab11's counter_driver had no analysis port. Lab12 adds a uvm_analysis_port so the driver can broadcast each stimulus transaction to the scoreboard before the DUT executes it:

Figure 4 — Only three lines changed in counter_driver (highlighted)

```
  // NEW in Lab12 — added to counter_driver
  uvm_analysis_port #(counter_transaction) ap;

  // build_phase — create the port
  ap = new("ap", this);

  // run_phase — publish BEFORE driving
  seq_item_port.get_next_item(req);
  ap.write(req);            // ← NEW: scoreboard gets stim at time 0
  drive_transaction(req);   //   DUT executes N cycles later
  seq_item_port.item_done();
```

5.2 Inline Pre-load for Compound Operations

Compound operations (COUNT_UP, COUNT_DOWN, OVERFLOW, UNDERFLOW) must pre-load a start value before counting. A naive approach calls the load_counter() interface task, but that task ends with an explicit MODE_HOLD cycle — which triggers the monitor. This creates a spurious response transaction that misaligns the scoreboard's FIFO queue.

Figure 5 — Inline pre-load avoids spurious LOAD→HOLD monitor trigger in compound ops

```
  WRONG — load_counter() causes a spurious monitor event:
    vif.load_counter(tr.start_count);   // LOAD → HOLD ← monitor fires here!
    vif.count_up(tr.up_cycles, ...);    // COUNT_UP → HOLD ← monitor fires
again
    // Result: 2 monitor events per COUNT_UP transaction → queue misaligned

  CORRECT — inline pre-load avoids the intermediate HOLD:
    vif.tb_cb.mode      <= MODE_LOAD;   // set LOAD directly
    vif.tb_cb.load_data <= tr.start_count;
    @(vif.tb_cb);                       // one LOAD cycle
    vif.count_up(tr.up_cycles, ...);    // LOAD → COUNT_UP → HOLD
    // Mode sequence: HOLD → LOAD → COUNT_UP×N → HOLD
    // Result: exactly 1 monitor event (at COUNT_UP→HOLD) ✓
```

5.3 Why Publish BEFORE Driving?

The scoreboard needs the stimulus fields (test_type, start_count, up_cycles…) to compute the EXPECTED result. It must know what to expect BEFORE the DUT finishes — otherwise it cannot verify the result when the monitor fires. The timeline below illustrates this:

Figure 6 — Stimulus arrives at scoreboard before DUT has finished executing

```
Simulation time ──────────────────────────────────────────────────────►

Time 0:  driver calls ap.write(req)
           └► scoreboard.write_stim(req)
                 └► compute_expected() → push exp_tr onto exp_q

Time 0+: drive_transaction(req) begins  ──► DUT starts executing

Time N:  DUT completes; mode → HOLD
           monitor fires: ap.write(obs)
             └► scoreboard.write_resp(obs)
                   └► pop exp_tr from exp_q
                      compare obs.act_count vs exp_tr.act_count
                      compare obs.act_ovf   vs exp_tr.act_ovf
                      compare obs.act_unf   vs exp_tr.act_unf

Rule: write_stim ALWAYS fires before write_resp
      (driver publishes at time 0; monitor fires after N clock cycles)
```

6. Scoreboard (counter_scoreboard.sv — NEW)

6.1 The Dual Analysis-Imp Pattern

A UVM component can only have ONE write() method by default. The scoreboard needs TWO write methods — one for stimulus (from the driver) and one for the observed response (from the monitor). The solution is the uvm_analysis_imp_decl macro:

Figure 7 — Macro placement and class declarations for dual-imp scoreboard

```
// These two lines appear at PACKAGE SCOPE (outside the class)
// They generate two new SV classes with suffixed write() dispatch:
`uvm_analysis_imp_decl(_stim)
// generates class uvm_analysis_imp_stim that calls parent.write_stim()

`uvm_analysis_imp_decl(_resp)
// generates class uvm_analysis_imp_resp that calls parent.write_resp()

class counter_scoreboard extends uvm_scoreboard;
  uvm_analysis_imp_stim #(counter_transaction, counter_scoreboard) stim_imp;
  uvm_analysis_imp_resp #(counter_transaction, counter_scoreboard) resp_imp;
  ...
  function void write_stim(counter_transaction tr); ... endfunction
  function void write_resp(counter_transaction obs); ... endfunction
```

```
  endclass
```

6.2 TEST_RESET Special Case — No Monitor Event

TEST_RESET is a special case that must be handled carefully. The driver calls apply_reset() which holds mode=HOLD throughout. The monitor only fires when mode TRANSITIONS from a non-HOLD mode to HOLD. Because mode starts at HOLD and never leaves HOLD during reset, NO monitor event fires. If write_stim() pushed an expected entry for TEST_RESET, the queue would accumulate a stale entry that would misalign all subsequent comparisons.

Figure 8 — write_stim skips TEST_RESET to keep the FIFO queue aligned

```
  write_stim(tr):
    // TEST_RESET: apply_reset() keeps mode=HOLD throughout.
    // Monitor fires only on non-HOLD → HOLD transitions.
    // No monitor event → no response → skip the queue push.
    if (tr.test_type == TEST_RESET) return;

    // All other test types: compute expected and enqueue
    compute_expected(tr, exp_tr.act_count, exp_tr.act_ovf, exp_tr.act_unf);
    exp_q.push_back(exp_tr);

  Monitor behaviour for TEST_RESET:
    prev_mode = HOLD (initial)     mode = HOLD (apply_reset keeps HOLD)
    Condition: mode==HOLD && prev_mode!=HOLD → FALSE → no fire ✓
```

6.3 Expected-Result FIFO Queue

The scoreboard maintains a queue of expected-result transactions. write_stim() pushes; write_resp() pops. Because the driver always publishes before the monitor, the queue maintains correct FIFO ordering:

Figure 7 — FIFO queue keeps stimulus and response in order

```
  exp_q[$]  (SystemVerilog queue — dynamic FIFO)

  Transaction 1 stimulus:  write_stim(tr1) → push exp_tr1  exp_q = [exp_tr1]
  Transaction 1 response:  write_resp(obs1) → pop  exp_tr1  exp_q = []
              compare obs1.act_count with exp_tr1.act_count  ✓ or ✗

  Transaction 2 stimulus:  write_stim(tr2) → push exp_tr2  exp_q = [exp_tr2]
  Transaction 3 stimulus:  write_stim(tr3) → push exp_tr3  exp_q = [exp_tr2,
exp_tr3]
  Transaction 2 response:  write_resp(obs2) → pop  exp_tr2  exp_q = [exp_tr3]
  Transaction 3 response:  write_resp(obs3) → pop  exp_tr3  exp_q = []
```

6.3 Golden Reference Model (compute_expected)

compute_expected() is the mathematical model of what the DUT should produce. It uses counter modular arithmetic:

test_type	Expected count	Expected ovf	Expected unf
TEST_RESET	0	0	0
TEST_LOAD	load_value	0	0
TEST_COUNT_UP	(start + up_cycles) % 256	1 if (start+N)%256==0	0
TEST_COUNT_DOWN	(start − down_cycles + K×256) % 256	0	1 if (start−N+1)%256==0
TEST_OVERFLOW	0	1	0
TEST_UNDERFLOW	255	0	1

Overflow fires on the LAST counting cycle only. For count_up: overflow = 1 iff the last step was 255→0. Mathematically: (start + up_cycles) % 256 == 0. This formula is correct even when the counter wraps multiple times, because it asks "does the final value land on 0?" — which is true only when the last step was a wrap.

7. Functional Coverage (counter_coverage.sv — NEW)

7.1 Code Coverage vs Functional Coverage

Code Coverage	Functional Coverage
Measures which RTL LINES / BRANCHES / TOGGLES were executed. Automatically collected by the simulator. Does NOT tell you which test scenarios were applied. Can reach 100% with a single directed test.	Measures which STIMULUS SCENARIOS were applied. Defined by the engineer via covergroups. Tells you whether all interesting combinations were tested. Requires rich stimulus (constrained-random) to hit all bins.

7.2 counter_coverage Covergroup

counter_coverage extends uvm_component (not uvm_subscriber) and declares its own analysis imp using the `uvm_analysis_imp_decl(_cov) macro. This approach avoids the argument-name constraint imposed by uvm_subscriber (which requires write(T t) with argument named "t") and is consistent with how the scoreboard's dual imps are declared. The coverage collector connects to the DRIVER's ap because stimulus transactions carry test_type, load_value, start_count, and cycle counts — the fields needed for rich coverpoint measurement.

Figure 8 — counter_cg covergroup definition

```
covergroup counter_cg;

  cp_test_type: coverpoint cv_test_type {
    bins reset     = {TEST_RESET};        // 1 of 6 required
    bins load      = {TEST_LOAD};
    bins count_up  = {TEST_COUNT_UP};
    bins count_dn  = {TEST_COUNT_DOWN};
    bins overflow  = {TEST_OVERFLOW};
    bins underflow = {TEST_UNDERFLOW};
  }

  cp_value: coverpoint cv_value {          // load_value or start_count
```

```
      bins zero = {0};                        // boundary
      bins low  = {[1:63]};                   // lower quarter
      bins mid  = {[64:191]};                 // middle half
      bins high = {[192:255]};                // upper quarter
    }

    cp_cycles: coverpoint cv_cycles {         // up_cycles / down_cycles
      bins one  = {1};                        // single step
      bins few  = {[2:4]};
      bins many = {[5:10]};
    }

    cx_type_val: cross cp_test_type, cp_value;  // 6 × 4 = 24 cross bins

  endgroup
```

Figure 9 — 24-bin cross coverage matrix

```
  Coverage bin matrix  (cx_type_val cross — 24 bins)

                   zero     low(1-63)  mid(64-191)  high(192-255)
  TEST_RESET      [  1  ]  [   1    ]  [    1     ]  [    1     ]
  TEST_LOAD       [  2  ]  [   2    ]  [    2     ]  [    2     ]
  TEST_COUNT_UP   [  3  ]  [   3    ]  [    3     ]  [    3     ]
  TEST_COUNT_DOWN [  4  ]  [   4    ]  [    4     ]  [    4     ]
  TEST_OVERFLOW   [  5  ]  [   5    ]  [    5     ]  [    5     ]
  TEST_UNDERFLOW  [  6  ]  [   6    ]  [    6     ]  [    6     ]

  Each cell must be hit at least once for 100% cross coverage.
  Only constrained-random stimulus can realistically hit all 24 bins.
```

7.3 Connectivity

Figure 10 — Coverage uses uvm_component + explicit analysis_imp (not uvm_subscriber)

```
  // Package scope (outside class) — generates uvm_analysis_imp_cov class
  `uvm_analysis_imp_decl(_cov)

  class counter_coverage extends uvm_component;
    uvm_analysis_imp_cov #(counter_transaction, counter_coverage)
analysis_imp;
    virtual function void write_cov(counter_transaction tr); ... endfunction
  endclass

  counter_env::connect_phase:
    agent.driver.ap.connect(cov.analysis_imp);
    //  ↑                        ↑
    //  uvm_analysis_port        uvm_analysis_imp_cov (explicit)
    //  (in counter_driver)      (declared in counter_coverage)

  Data flow:
    driver.ap.write(req)  ──►  cov.analysis_imp  ──►  cov.write_cov(req)
```

```
                                                        write_cov() samples
the covergroup
```

8. Virtual Sequencer (counter_virtual_sequencer.sv — NEW)

8.1 Real Sequencer vs Virtual Sequencer

Real Sequencer (agent.sequencer)	Virtual Sequencer (env.vseqr)
Connected to a driver via TLM seq_item_port. Receives get_next_item() calls from the driver. Can only serve one type of transaction. Controls a single DUT interface.	NOT connected to any driver — never generates transactions. Holds HANDLES to one or more real sequencers. Virtual sequences start sub-sequences through these handles. Coordinates activity across multiple real sequencers.

8.2 Handle Assignment

Figure 11 — Virtual sequencer handle assigned in connect_phase

```
  counter_virtual_sequencer (only holds a handle — creates nothing):
    uvm_sequencer #(counter_transaction) counter_seqr;  // handle only

  counter_env::connect_phase (assigns the handle):
    vseqr.counter_seqr = agent.sequencer;
    //                ↑               ↑
    //      vseqr's handle   real sequencer owned by the agent

  counter_virtual_sequence::body() (uses the handle):
    $cast(vseqr, m_sequencer);         // get virtual seqr handle
    dir_seq.start(vseqr.counter_seqr); // start on the REAL sequencer
```

9. Virtual Sequence (counter_virtual_sequence.sv — NEW)

9.1 How the Virtual Sequence Pattern Works

A virtual sequence runs on the virtual sequencer but drives the DUT indirectly by starting real sub-sequences on the real (agent) sequencer. The critical step is casting m_sequencer to counter_virtual_sequencer to access the counter_seqr handle:

Figure 12 — Virtual sequence orchestrates sub-sequences on real sequencer

```
  counter_virtual_test::run_phase:
    vseq = counter_virtual_sequence::type_id::create("vseq");
    vseq.start(env.vseqr);     ← started on VIRTUAL sequencer

  counter_virtual_sequence::body():
    counter_virtual_sequencer vseqr;
    $cast(vseqr, m_sequencer); ← m_sequencer IS env.vseqr
```

```
  dir_seq = counter_sequence::type_id::create("dir_seq");
  dir_seq.start(vseqr.counter_seqr);  ← runs on REAL sequencer
  // dir_seq.start() blocks until all 6 transactions are done

  rnd_seq = counter_random_sequence::type_id::create("rnd_seq");
  rnd_seq.start(vseqr.counter_seqr);  ← runs on same REAL sequencer
  // rnd_seq.start() blocks until all 20 random transactions are done
```

9.2 Execution Timeline

Figure 13 — Virtual sequence execution: directed phase then random phase

```
counter_virtual_test  ──▶  start(env.vseqr)
                                  │
                  counter_virtual_sequence::body()
                                  │
             ┌────────────────────┘
             │  Phase 1: dir_seq.start(vseqr.counter_seqr)
             │    ├── TEST_RESET
             │    ├── TEST_LOAD 42
             │    ├── TEST_COUNT_UP   (start=10, N=3)
             │    ├── TEST_COUNT_DOWN (start=20, N=2)
             │    ├── TEST_OVERFLOW
             │    └── TEST_UNDERFLOW
             │         ↓  (returns when all 6 done)
             │  Phase 2: rnd_seq.start(vseqr.counter_seqr)
             │    ├── random transaction 1
             │    ├── random transaction 2
             │    ├── ...
             │    └── random transaction 20
             │         ↓  (returns when all 20 done)
             └────  body() returns  ──▶  drop_objection
```

10. Factory Type Override (counter_random_test.sv — NEW)

10.1 What the Factory Does

Every UVM class that uses `uvm_object_utils or `uvm_component_utils is registered with the UVM factory. When code calls type_id::create(), the factory checks whether a type override is registered and returns the OVERRIDE type if one is found — without the calling code knowing.

Figure 14 — Factory override replaces directed with random; run_phase code unchanged

```
Lab11 / counter_test:                      counter_random_test:
──────────────────────                     ────────────────────────
run_phase:                                 build_phase:
  seq = counter_sequence                     counter_sequence::type_id
          ::type_id::create("seq");            ::set_type_override(
  seq.start(env.agent.sequencer);                counter_random_sequence
                                                   ::get_type());
                                             super.build_phase(phase);
Factory lookup:                                        │
```

```
    "create counter_sequence"                                  ▼
     → override registered?                          run_phase (inherited):
     → YES → return counter_random_sequence             seq = counter_sequence

::type_id::create("seq");
                                                        // factory returns
                                                        //
counter_RANDOM_sequence!
```

10.2 Type Override vs Instance Override

Global Type Override	Instance Override
set_type_override(new_type) Replaces EVERY creation of the original type anywhere in the entire testbench. Used in counter_random_test.	set_inst_override(new_type, path_glob) Replaces only creations from components whose hierarchical path matches the glob pattern. Useful when only one agent should use the override.

11. Constrained-Random Sequence (counter_random_sequence.sv — NEW)

11.1 Why Constrained-Random?

The directed counter_sequence covers exactly 6 predetermined scenarios. A constrained-random sequence generates N transactions automatically, each with randomised fields. Constraints prevent illegal combinations (e.g., up_cycles must be ≥ 1 for TEST_COUNT_UP) while the simulator explores the entire legal stimulus space.

11.2 Config-Driven Transaction Count

Figure 15 — Random sequence reads num_transactions from config and loops

```
  counter_random_sequence::body():
    // Read how many transactions to generate from config
    if (uvm_config_db#(counter_config)::get(
          get_sequencer(), "", "cfg", cfg))
      num_trans = cfg.num_transactions;   // use configured value
    else
      num_trans = 20;                     // fall back to default

    repeat (num_trans) begin
      tr = counter_transaction::type_id::create("tr");
      start_item(tr);
      if (!tr.randomize())                // use base constraints
        `uvm_fatal("RNDSEQ","randomize() failed");
      finish_item(tr);
    end
```

12. Environment (counter_env.sv — UPDATED)

12.1 build_phase Changes

Lab11's counter_env created only the agent. Lab12 adds four more creation steps:

Figure 16 — Lab11 vs Lab12 env build_phase

```
  Lab11 build_phase:                  Lab12 build_phase:
  ──────────────────                  ──────────────────
                                      // Step 1: retrieve config (FATAL if
absent)
                                      if (!config_db::get(..., "cfg", cfg))
                                        `uvm_fatal("ENV", "config not found")

  agent = counter_agent               agent = counter_agent
            ::type_id::create(...)              ::type_id::create(...)

                                      // Step 2: create scoreboard (always)
                                      sb    =
counter_scoreboard::type_id::create(...)

                                      // Step 3: create virtual seqr (always)
                                      vseqr =
counter_virtual_sequencer::type_id::create(...)

                                      // Step 4: coverage only if enabled
                                      if (cfg.enable_coverage)
                                        cov =
counter_coverage::type_id::create(...)
```

12.2 connect_phase Changes

Lab11's counter_env had no connect_phase. Lab12 adds five connections:

Figure 17 — Lab12 connect_phase wires all analysis paths

```
  counter_env::connect_phase (NEW in Lab12):

  // 1. Driver → scoreboard: deliver stimulus for expected computation
  agent.driver.ap.connect(sb.stim_imp);

  // 2. Monitor → scoreboard: deliver response for comparison
  agent.monitor.ap.connect(sb.resp_imp);

  // 3. Driver → coverage: deliver stimulus for coverpoint sampling
  if (cfg.enable_coverage)
    agent.driver.ap.connect(cov.analysis_imp);

  // 4. Virtual sequencer: assign real sequencer handle
  vseqr.counter_seqr = agent.sequencer;

  Note: analysis ports support fan-out.
```

```
  agent.driver.ap is connected to BOTH sb.stim_imp and cov.analysis_imp.
  Both subscribers receive every write() call automatically.
```

13. Complete Data-Flow Diagram

The diagram below shows how transactions flow through the testbench for a single counter operation from sequence to scoreboard verdict:

Figure 18 — Complete transaction data flow through Lab12 testbench

```
  ┌──────────────────┐
  │  counter_sequence │  (or random_sequence / virtual_sequence)
  │  body():          │
  │   start_item(tr)  │
  │   finish_item(tr) │
  └──────────────────┘
          │ TLM: seq_item_export / seq_item_port
          ▼
  ┌──────────────────┐
  │ counter_sequencer │ (arbitration / scheduling)
  └──────────────────┘
          │ TLM: get_next_item(req)
          ▼
  ┌──────────────────────────────────────────────────────────┐
  │  counter_driver                                          │
  │   1. ap.write(req) ──────────────────────────────────┐   │
  │   2. drive_transaction(req) ──▶ counter_if tasks     │   │
  │   3. item_done()                                     │   │
  └──────────────────────────────────────────────────────┴───┘
                          │ DUT executes N cycles          │
analysis_port.write()
                          ▼                                │
  ┌──────────────────┐    │
├──────────────────────┐
  │  counter DUT     │    │                               ▼
▼
  │  (RTL)           ├────┘                ┌──────────────────┐
┌──────────────────┐
  └──────────────────┘                     │counter_scoreboard │
│counter_coverage  │
                                           │ write_stim(req):  │    │ write(req):
│
                                           │  compute_expected │    │  sample
cv_*     │
                                           │  push to exp_q    │    │
cg.sample()     │
  DUT outputs stable → mode→HOLD           └──────────┬───────┘
└──────────────────┘
          │                                           │
          ▼                                           │ (later, when monitor
fires)
  ┌──────────────────┐                                │
```

```
│  counter_monitor │                                  ▼
│  ap.write(obs)──┼──► sb.resp_imp ──► write_resp(obs):
└─────────────────┘                       pop exp_tr from exp_q
                                          compare act_count, ovf, unf
                                          PASS or FAIL message
```

14. Test Selection and Run Commands

14.1 How Test Selection Works

In Lab11, run_test("counter_test") hard-coded the test name. In Lab12, run_test() is called without arguments. The simulator passes +UVM_TESTNAME to select the test at runtime — no recompilation needed:

Figure 19 — run_test() with +UVM_TESTNAME allows runtime test selection

```
Lab11  counter_tb_top:          Lab12  counter_tb_top:
  run_test("counter_test");       run_test();  // test from +UVM_TESTNAME

Command-line selection:
  vsim ... +UVM_TESTNAME=counter_test
  vsim ... +UVM_TESTNAME=counter_random_test
  vsim ... +UVM_TESTNAME=counter_virtual_test
```

14.2 Test Comparison

Test	Sequence Used	Transactions	Scoreboard	Coverage
counter_test	counter_sequence (directed)	6 fixed	Yes	Yes (if enable_cov=1)
counter_random_test	counter_random_sequence (factory override)	N random (default 20)	Yes	Yes
counter_virtual_test	counter_virtual_sequence → directed + random	6 + N	Yes	Yes

14.3 Makefile Usage

```
# Directed test (default)
make questa_batch
make questa_batch TEST=counter_test

# Random stimulus test (factory override)
make questa_batch TEST=counter_random_test

# Virtual sequence test
make questa_batch TEST=counter_virtual_test

# GUI mode for waveform viewing
make questa_gui TEST=counter_virtual_test

# Clean build artifacts
make clean
```

15. Package Include Order (counter_pkg.sv — UPDATED)

Every class must be declared before it can be referenced. The include order in counter_pkg.sv follows a strict dependency chain:

Figure 20 — Dependency chain determines include order in counter_pkg.sv

```
 counter_pkg.sv  include order and dependency chain:

 1. counter_transaction       (no TB dependencies — leaf node)
      ↓ used by driver, monitor, agent, sequences
 2. counter_driver            (uses counter_transaction, counter_if)
 3. counter_monitor           (uses counter_transaction)
 4. counter_agent             (uses driver, monitor)

 5. counter_config            (uses uvm_object only — no TB deps)
      ↓ used by env, test, random_sequence

 6. counter_scoreboard        (uses counter_transaction)
      Note: `uvm_analysis_imp_decl macros inside this file expand
            at PACKAGE SCOPE before the class — this is correct.

 7. counter_coverage          (uses counter_transaction, test_type_t)
 8. counter_virtual_sequencer (uses counter_transaction via seqr type)

 9. counter_env               (uses agent, sb, cov, vseqr, config)
      ↓ env must come after ALL components it creates

10. counter_sequence          (uses counter_transaction)
11. counter_random_sequence   (extends counter_sequence)
12. counter_virtual_sequence  (uses counter_virtual_sequencer,
                                   counter_sequence, counter_random_sequence)

13. counter_test              (uses counter_env, counter_config)
14. counter_random_test       (extends counter_test)
                              (uses counter_random_sequence::get_type())
15. counter_virtual_test      (extends counter_test)
                              (uses counter_virtual_sequence)
```

16. Summary of Lab12 UVM Concepts

UVM Concept	Counter TB Component	Key API / Pattern
Configuration Object	counter_config	uvm_object; do_copy; do_print; config_db::set / get
Analysis Port (producer)	counter_driver.ap	uvm_analysis_port; ap.write() before driving
Analysis Imp (consumer)	counter_scoreboard.stim_imp counter_scoreboard.resp_imp	`uvm_analysis_imp_decl; write_stim() / write_resp()
uvm_component +	counter_coverage	extends uvm_component;

analysis_imp		`uvm_analysis_imp_decl(_cov); write_cov(); covergroup.sample()
Covergroup	counter_cg (in coverage)	coverpoint bins; cross; cg.get_coverage()
Virtual Sequencer	counter_virtual_sequencer	extends uvm_sequencer; holds counter_seqr handle
Virtual Sequence	counter_virtual_sequence	extends uvm_sequence; $cast(vseqr, m_sequencer); sub_seq.start(vseqr.handle)
Factory Type Override	counter_random_test	set_type_override() before super.build_phase(); get_type()
Test Selection	counter_tb_top	run_test(); +UVM_TESTNAME=test_name

17. Expected Simulation Output

When any of the three tests runs successfully, the UVM log will contain:

```
UVM_INFO counter_driver.sv  "Driving: test_type=TEST_LOAD load=42 ..."
UVM_INFO counter_monitor.sv "Observed: count=42 ovf=0 unf=0"
UVM_INFO sb                 "PASS count: got 42, expected 42"
UVM_INFO sb                 "PASS ovf:   got 0,  expected 0"
UVM_INFO sb                 "PASS unf:   got 0,  expected 0"
...
UVM_INFO SCOREBOARD  "=== Scoreboard Summary: 18 PASS, 0 FAIL ==="
UVM_INFO COV         "=== Functional Coverage: 87.5% ==="

UVM_FATAL if:  counter_config not in config_db
UVM_ERROR if:  act_count does not match expected
UVM_ERROR if:  response received but expected queue is empty
```

At end-of-elaboration, uvm_top.print_topology() prints the complete component hierarchy showing all ports, exports, and connections. This is invaluable for verifying that analysis paths are wired correctly.